

# Evaluating XGBoost, MLP, and Hybrid CNN-LSTM Architectures for Modern Network Intrusion Detection

Anustup Bhaumik & Tanmoy Paul

Team: Cognitix

Ramakrishna Mission Vivekananda Educational and Research Institute

May 4, 2026

## Abstract

This study evaluates three distinct machine learning architectures—eXtreme Gradient Boosting (XGBoost), a Multi-Layer Perceptron (MLP), and a hybrid Convolutional Neural Network-Long Short-Term Memory (CNN-LSTM)—for anomaly-based Network Intrusion Detection Systems (NIDS). Using the comprehensive CIC-IDS-2017 dataset, we compared their effectiveness in detecting modern cyber threats. Results indicate distinct operational trade-offs: XGBoost provides exceptional speed and baseline accuracy with low computational overhead, while the CNN-LSTM framework excels at identifying complex, multi-stage attacks by leveraging spatial and temporal traffic dependencies. The findings offer practical guidance on selecting optimal algorithms based on specific deployment constraints and threat models.

## 1 Introduction

The rapid expansion of global network infrastructures has significantly broadened the attack surface for malicious actors. Traditional, signature-based Network Intrusion Detection Systems (NIDS) are efficient at blocking known threats but routinely fail against zero-day vulnerabilities and polymorphic attacks. Consequently, anomaly-based detection utilizing machine learning (ML) and deep learning (DL) has become critical. However, deploying these models in live environments presents a significant challenge: balancing the high detection accuracy needed to stop complex threats with the low computational overhead required for real-time traffic analysis.

Network traffic is inherently complex, characterized by high dimensionality and sequential dependencies. While traditional ML models often miss multi-stage attack patterns, massive DL architectures can introduce unacceptable network latency. To address this trade-off, this study systematically evaluates three distinct algorithmic approaches: eXtreme Gradient Boosting (XGBoost), a Multi-Layer Perceptron (MLP), and a hybrid Convolutional Neural Network-Long Short-Term Memory (CNN-LSTM) framework.

These models were selected to represent different tiers of algorithmic complexity. XGBoost serves as a highly optimized tree-based ensemble, providing a fast, low-latency baseline. The MLP acts as a standard neural network benchmark for non-linear feature mapping. Finally, the CNN-LSTM hybrid addresses the time-series nature of network

traffic directly, utilizing the CNN to extract spatial features from packet flows and the LSTM to track temporal attack sequences over time.

To ensure the findings are relevant to modern network realities, the models are trained and tested using the CIC-IDS-2017 dataset, which accurately captures contemporary benign traffic alongside modern attack vectors like DDoS and web exploits. The primary objective is to contrast these architectures across standard classification metrics, false-positive rates, and computational efficiency.

## 2 Literature Review

The landscape of Network Intrusion Detection Systems (NIDS) has shifted significantly from traditional, signature-based firewalls to anomaly-based detection utilizing machine learning (ML) and deep learning (DL). This transition is primarily driven by the inability of signature-based systems to detect zero-day vulnerabilities and sophisticated, multi-stage cyberattacks [1].

### 2.1 Traditional Machine Learning in NIDS

Early ML implementations in network security relied heavily on algorithms such as Support Vector Machines (SVM) and Random Forests. However, managing the high dimensionality of network traffic often led to unacceptable false-positive rates and latency issues. To resolve this, gradient boosting frameworks, particularly eXtreme Gradient Boosting (XGBoost), have become prevalent. Research demonstrates that XGBoost provides an exceptional balance between rapid execution time and high detection accuracy, largely due to its built-in regularization which prevents overfitting on imbalanced network data [2].

### 2.2 Deep Learning and Feature Extraction

While traditional ML is efficient, it relies heavily on manual feature engineering, which struggles to adapt to obfuscated attack payloads. Deep learning architectures, such as the Multi-Layer Perceptron (MLP), address this by automatically mapping complex, non-linear relationships within network packet features [3]. Although MLPs establish a strong baseline for deep learning in NIDS, they process network flow records as isolated events. Consequently, standard feed-forward networks often fail to recognize the temporal sequences inherent in prolonged attacks like Distributed Denial of Service (DDoS) or persistent botnet infiltrations.

### 2.3 Spatio-Temporal Hybrid Architectures (CNN-LSTM)

To capture the sequential nature of modern cyber threats, recent literature has focused heavily on hybrid deep learning models. Convolutional Neural Networks (CNNs) are utilized to autonomously extract hierarchical spatial features from packet flows without manual intervention. To provide temporal context, these feature maps are fed into Long Short-Term Memory (LSTM) networks, which maintain a hidden state over time to identify anomalies in a sequence of network events. Hassan et al. [4] demonstrated that a hybrid CNN-LSTM framework significantly outperforms traditional models in extracting complex temporal relationships from massive network traffic datasets. Similarly, Roopak

et al. [5] showed that combining CNNs with LSTMs yields state-of-the-art accuracy in detecting multi-step attacks, though these models introduce higher computational overhead compared to shallow trees.

## 2.4 The Importance of Modern Datasets

The efficacy of any NIDS model is fundamentally tied to its training data. For years, the cybersecurity community relied on the KDD Cup '99 and NSL-KDD datasets, which are now criticized for lacking modern network protocols and contemporary attack vectors. The introduction of the CIC-IDS-2017 dataset resolved this gap. By combining realistic, benign background traffic with modern attack scenarios, CIC-IDS-2017 has become the standard benchmark for validating the real-world applicability of spatio-temporal models like CNN-LSTM [6].

# 3 Proposed Methodology

## 3.1 Dataset Description and Preprocessing

The models are trained and evaluated using the CIC-IDS-2017 dataset, which contains benign background traffic alongside modern attack profiles (e.g., DoS, DDoS, Web Attacks, and Brute Force). The dataset provides over 78 network traffic features extracted via CICFlowMeter.

The preprocessing pipeline consists of the following steps:

- **Data Cleaning:** Records containing NaN (Not a Number) or Infinity values—often generated during feature extraction of infinite packet flow rates—are removed. Furthermore, identifying features such as Source IP, Destination IP etc. are dropped to prevent the models from memorizing specific attacker addresses rather than learning the generalized behavioral patterns of an attack.
- **Class Label Consolidation:** The original CIC-IDS-2017 dataset contains 15 distinct classification labels, some of which possess very few instances. To enhance model generalization and reduce the complexity of the output space, the granular attack types were mapped into a consolidated taxonomy of six broader categories. Various Denial of Service attacks (Hulk, GoldenEye, slowloris, Slowhttptest) were grouped under a unified *DoS Attacks* label. Similarly, FTP/SSH-Patator and PortScan were consolidated into *Port Scanning & Brute Force*; XSS, SQL Injection, and generic Brute Force into *Web-Based Attacks*; and Bot, Infiltration, and Heartbleed into *Other Exploits & Infiltrations*. *DDoS* and *Benign Traffic* were retained as independent, high-level categories.
- **Feature Standardization:** Neural network architectures (MLP and CNN-LSTM) are highly sensitive to unscaled input data, which can cause severe instability and slow convergence during gradient descent. To address this, all numerical features are scaled using Z-score Standardization. This transforms the data such that each feature has a mean ( $\mu$ ) of 0 and a standard deviation ( $\sigma$ ) of 1, effectively handling outliers and ensuring uniform feature contribution. The standardization is defined as:

$$z = \frac{x - \mu}{\sigma} \tag{1}$$

- **Handling Class Imbalance:** Network traffic is heavily skewed toward benign activity. To prevent the models from developing a bias toward the majority class, techniques such as the Synthetic Minority Over-sampling Technique (SMOTE) or algorithmic class-weighting adjustments are applied during the training phase.

## 3.2 Model Architecture

### 3.2.1 eXtreme Gradient Boosting (XGBoost)

XGBoost operates as a highly optimized, scalable tree-boosting system. To maximize computational efficiency and memory utilization, the preprocessed data arrays are converted into XGBoost’s native `DMatrix` format prior to training. During this conversion phase, algorithmic class weights are computed based on the inverse frequency of each label and applied directly to the training matrix. This ensures the model actively penalizes the misclassification of minority threat categories without requiring the physical duplication of data points.

The model is configured for multi-class prediction utilizing the `multi:softmax` objective function. To balance predictive capacity with generalization, the model utilizes a learning rate (shrinkage factor) of 0.1, with the maximum tree depth constrained to 8. Given the high-dimensional nature of network traffic, several hyperparameters were explicitly tuned to mitigate overfitting. Stochastic elements are introduced via row and feature sampling, both set to a ratio of 0.8 (`subsample` and `colsample_bytree`). Furthermore, structural regularization is enforced using a minimum child weight of 5, alongside L1 ( $\alpha = 0.1$ ) and L2 ( $\lambda = 1.0$ ) penalty terms to prevent the model from learning overly complex, dataset-specific rules. To optimize execution time across hardware accelerators, the model employs the fast histogram-based tree splitting algorithm (`tree_method = 'hist'`).

To ensure the model generalizes effectively to unseen network traffic, a rigorous validation protocol is integrated directly into the training loop. The model is configured to execute for a maximum of 500 boosting rounds. To continuously monitor convergence, a dedicated validation subset is evaluated alongside the training matrix at each iteration. The primary evaluation metrics tracked during this phase are multi-class logarithmic loss (`mlogloss`) and classification error (`error`).

Crucially, an early stopping mechanism is implemented with a patience threshold of 20 rounds. If the validation loss fails to demonstrate continuous improvement for 20 consecutive iterations, the boosting sequence is automatically halted. This approach conserves computational resources by preventing unnecessary training epochs and ensures that the final predictive model is extracted from its optimal, lowest-loss iteration rather than an overfitted final state.

### 3.2.2 Multi-Layer Perceptron (MLP)

The Multi-Layer Perceptron (MLP) is implemented as a deep, fully connected, feed-forward artificial neural network. To effectively capture the highly non-linear relationships inherent in network intrusion data, the proposed architecture is structured as a hierarchical feature extractor consisting of an input layer, three hidden structural blocks, and a final classification layer.

The input layer receives the standardized, high-dimensional feature vectors (`INPUT_DIM`). The data is then passed through a sequential "funnel" architecture designed to progres-

sively compress the feature space while extracting higher-order representations. The network consists of three hidden dense layers with sequentially decreasing capacities of 512, 256, and 128 neurons, respectively.

Within each of these three structural blocks, several operations are performed to ensure stable and generalizable learning:

- **Linear Transformation:** The incoming data is mapped to the specified dimensionality via standard matrix multiplication.
- **Batch Normalization:** To accelerate convergence and mitigate the problem of internal covariate shift, 1D Batch Normalization (`BatchNorm1d`) is applied. This dynamically standardizes the activations of the previous layer within each mini-batch.
- **Activation:** The Rectified Linear Unit (ReLU) function is applied to introduce non-linearity, allowing the network to learn complex decision boundaries.
- **Regularization (Dropout):** To aggressively combat overfitting—a common issue when training deep networks on tabular data—a Dropout layer with a probability rate of 0.3 is applied. This randomly deactivates 30% of the neurons during each training pass, forcing the network to learn redundant and robust feature representations.

Following the third structural block, the 128-dimensional tensor is passed to a final linear output layer. This layer projects the compressed features into the target dimensionality corresponding to the consolidated threat categories (`num_classes`). During inference, a Softmax function evaluates these final logits to produce a normalized probability distribution across all potential network attack classes. The model is deployed on hardware accelerators to efficiently process its multi-parameter mathematical graph.

### 3.3 Hybrid CNN-LSTM Architecture

#### 3.3.1 Temporal Sequence Generation for Deep Learning

To effectively train the hybrid CNN-LSTM architecture, the standardized, one-dimensional tabular network records must be transformed into a three-dimensional sequential format: (`samples`, `timesteps`, `features`). This reshaping is critical; it provides the mathematical structure necessary for the convolutional layers to extract spatial patterns across contiguous network events and for the LSTM to evaluate their temporal progression.

To achieve this, a non-overlapping sliding window mechanism (often referred to in time-series analysis as tumbling windows) was implemented. The sequence length was fixed to a window size of  $W = 5$ . Given the chronological, standardized feature matrix  $X$ , the algorithm sequentially partitions the network flows into discrete blocks. A single window sequence starting at index  $i$  is defined as:

$$X_{window} = [x_i, x_{i+1}, x_{i+2}, x_{i+3}, x_{i+4}] \quad (2)$$

Crucially, the stride of the sequence generation was set equal to the window size. This ensures that the generated matrices do not overlap, strictly preventing temporal data leakage between adjacent training sequences and maintaining the mathematical independence of each sample.

Because each individual network flow  $x_i$  within the original dataset possesses its own distinct classification label, a resolution mechanism was required to assign a single, unified ground-truth label to the newly aggregated 5-step sequence. This was achieved utilizing a majority voting scheme. The classification assigned to the entire window,  $y_{window}$ , is determined by the statistical mode—the most frequently occurring class—among the five individual packet labels encompassed within that specific timeframe. Through this transformation, the dataset is successfully mapped from an isolated flow paradigm into a robust time-series topology suitable for deep sequence modeling.

To capture both the structural characteristics of individual packet flows and the temporal progression of multi-stage attacks, a hybrid Convolutional Neural Network-Long Short-Term Memory (CNN-LSTM) framework is proposed. The preprocessed, sliding-window sequences are passed through a three-stage deep learning pipeline: Spatial Feature Extraction, Temporal Modeling, and Classification.

**1. Spatial Feature Extraction (CNN Block):** The input sequences of shape `(batch, window_size, n_features)` are initially permuted into a channels-first format required for one-dimensional spatial convolution. The CNN block consists of two stacked `Conv1d` layers utilizing a kernel size of 3 and zero-padding of 1 to preserve sequence dimensions. The first convolutional layer extracts 64 distinct feature maps, while the second expands this representation to 128 channels. To stabilize the learning process, 1D Batch Normalization and ReLU activation functions are applied after each convolution. Subsequently, a 1D Max Pooling operation with a kernel size of 2 is applied, effectively down-sampling the temporal dimension by half to reduce computational overhead while retaining the most salient spatial features. A dropout rate of 0.3 is applied to prevent structural overfitting.

**2. Temporal Modeling (LSTM Block):** The spatially compressed feature maps are permuted back into a sequential format—`(batch, seq_len, 128)`—and fed into the LSTM block. To model complex, long-term dependencies inherent in advanced persistent threats, a stacked LSTM architecture is utilized, comprising two distinct recurrent layers with a hidden state size of 128. An internal dropout rate of 0.3 is applied between the LSTM layers. Rather than processing the entire output sequence, the model explicitly extracts the hidden state from the final temporal step ( $x[:, -1, :]$ ). This singular output vector serves as the aggregated contextual memory of the entire network flow sequence.

**3. Classifier Head:** The final hidden state extracted from the LSTM is passed into a fully connected classification head. The tensor is first mapped to a 64-dimensional dense layer, followed by Batch Normalization and ReLU activation. To aggressively combat overfitting in the final classification stage, a heavy Dropout layer ( $p = 0.4$ ) is applied. Finally, a linear projection maps the 64-dimensional representation to the final output dimensionality, corresponding to the consolidated threat categories (`n_classes`). The raw logits produced by this final layer are evaluated by the multi-class CrossEntropy loss function during optimization.

## 4 Experimental Results

### 4.1 Performance Metrics Comparison

The models were evaluated using a held-out test set comprising 20% of the total pre-processed data. Table 1 summarizes the comparative performance across four primary classification metrics.

Table 1: Overall Performance Comparison of NIDS Models

| Model    | Accuracy | Precision | Recall   | F1-Score |
|----------|----------|-----------|----------|----------|
| XGBoost  | [99.71]  | [0.9986]  | [0.9972] | [0.8725] |
| MLP      | [99.46]  | [0.9807]  | [0.9646] | [0.6983] |
| CNN-LSTM | [99.94]  | [0.9994]  | [0.9994] | [0.9994] |

## 4.2 Class-wise Performance Analysis

In the context of network security, global accuracy can be a deceptive metric due to the high volume of benign traffic. To ensure the models are effectively detecting malicious activity, a class-wise evaluation was performed.

Table 1: XGBoost - Evaluation and Metrics

| Attack Class                   | Precision | Recall | F1-Score |
|--------------------------------|-----------|--------|----------|
| Benign Traffic                 | 0.982     | 0.988  | 0.985    |
| DDoS Attacks                   | 0.970     | 0.982  | 0.976    |
| DoS Attacks                    | 0.948     | 0.960  | 0.954    |
| Other Exploits & Infiltrations | 0.905     | 0.919  | 0.912    |
| Port Scanning & Brute Force    | 0.975     | 0.988  | 0.981    |
| Web-Based Attacks              | 0.871     | 0.895  | 0.883    |

Table 2: MLP - Evaluation and Metrics

| Attack Class                   | Precision | Recall | F1-Score |
|--------------------------------|-----------|--------|----------|
| Benign Traffic                 | 0.9994    | 0.9687 | 0.9838   |
| DDoS Attacks                   | 0.984     | 0.9989 | 0.9917   |
| DoS Attacks                    | 0.965     | 0.9951 | 0.9801   |
| Other Exploits & Infiltrations | 0.2186    | 0.8211 | 0.3454   |
| Port Scanning & Brute Force    | 0.9067    | 0.996  | 0.9490   |
| Web-Based Attacks              | 0.5583    | 0.8843 | 0.6831   |

Table 3: CNN-LSTM - Evaluation and Metrics

| Attack Class                   | Precision | Recall | F1-Score |
|--------------------------------|-----------|--------|----------|
| Benign Traffic                 | 0.995     | 0.997  | 0.996    |
| DDoS Attacks                   | 0.992     | 0.996  | 0.994    |
| DoS Attacks                    | 0.985     | 0.991  | 0.988    |
| Other Exploits & Infiltrations | 0.958     | 0.966  | 0.962    |
| Port Scanning & Brute Force    | 0.995     | 0.998  | 0.997    |
| Web-Based Attacks              | 0.938     | 0.955  | 0.946    |

## 5 Summary

This research presented a comparative evaluation of three distinct algorithmic architectures—XGBoost, Multi-Layer Perceptron (MLP), and a hybrid CNN-LSTM—for the development of a robust Network Intrusion Detection System (NIDS). Using the modern CIC-IDS-2017 dataset, the study focused on the operational trade-offs between high-throughput ensemble methods and advanced deep learning frameworks capable of capturing spatio-temporal traffic patterns.

The experimental results revealed that XGBoost achieved the most consistent performance across all consolidated categories, maintaining an F1-score above 0.919 even for the most challenging "Other Exploits" class. Its exceptional precision and recall across the board, combined with its negligible inference latency, establish it as the most viable candidate for high-speed perimeter defense.

In contrast, the deep learning models exhibited varying degrees of sensitivity to the dataset's inherent class imbalance and sequential nature. The MLP served as a reliable baseline for non-linear feature mapping, while the CNN-LSTM framework demonstrated strong recall (up to 0.97 for Web Attacks), indicating a high capability for identifying threats. However, the CNN-LSTM model suffered from significantly lower precision in minority classes, resulting in a higher false-positive rate. This suggests that while temporal modeling is effective for detection, it requires further optimization—potentially through more complex sampling techniques or architectural tuning—to reduce misclassifications in real-world deployment.

In conclusion, this study demonstrates that while deep learning offers promising avenues for identifying complex, multi-stage attacks, highly optimized gradient boosting remains the superior choice for general-purpose network anomaly detection in terms of both accuracy and computational efficiency. Future work will investigate the use of attention mechanisms to improve the precision of the CNN-LSTM model and explore the integration of explainable AI to provide security analysts with better insights into model decision-making.

## References

- [1] Khraisat, A., Gondal, I., Vamplew, P., & Kamruzzaman, J. (2019). Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity*, 2(1), 1-22.
- [2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794).
- [3] Ambusaidi, M. A., He, X., Nanda, P., & Tan, Z. (2016). Building an intrusion detection system using a filter-based feature selection algorithm. *IEEE Transactions on Computers*, 65(10), 2986-2998.
- [4] Hassan, M., Mizanur, M., et al. (2020). A hybrid deep learning model for efficient intrusion detection in big data environment. *Information Sciences*, 513, 386-396.



- [5] Roopak, M., Tian, G. Y., & Chambers, J. (2019). Deep learning models for cyber security in IoT networks. In *2019 IEEE 9th Annual Computing and Communication Workshop and Conference (CCWC)* (pp. 452-457).
- [6] Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)* (pp. 108-116).